

Claude Code

Table of Contents

Claude Code	1
Start session	4
Essential commands.....	4
Pro tips	4
The agentic loop.....	5
Tools	5
Work with sessions	6
Work across branches	6
Resume or fork sessions	6
Manage context with skills and subagents.....	6
Checkpoints and permissions.....	7
Undo changes with checkpoints	7
Work effectively with Claude Code	7
Extend Claude Code.....	7
Match features to your goal	8
Claude.md	8
Load from additional directories.....	9
Organize rules with .claude/rules/	9
Write effective instructions	10
Import additional files	11
View and edit with /memory	11
Manage CLAUDE.md for large teams	11
Claude.md Locations.....	13
Auto memory.....	13
Storage Location.....	14
How it works	14

Configure permissions.....	14
CLI Tools	14
Skills.....	15
MCP	16
Subagents.....	16
Use specialized subagents	17
Agent teams	18
Hooks.....	19
Set up hooks.....	19
Plugins	19
Understand how features layer.....	19
Understand context costs.....	20
Context cost by feature.....	20
Understand how features load	20
Combine features.....	21
Plan Mode	22
When to use Plan Mode	22
Explore first, then plan, then code	22
Configure Plan Mode as default.....	23
Extended thinking	23
Adaptive thinking	23
Adaptive thinking with the effort parameter.....	24
Session.....	24
Resume previous conversations.....	24
Name your sessions	25
Use the session picker.....	25
Run parallel Claude Code sessions with Git worktrees	26
Subagent worktrees.....	26
Worktree cleanup.....	26
Manage worktrees manually.....	27
Create a worktree with a new branch	27
Common Workflows	27
Working with Tests.....	27
Create pull requests.....	27
Reference files and directories.....	27
Pipe in, pipe out.....	28

Best Practices.....	29
Give cc a way to verify its work.....	29
Provide specific context in your prompts	29
Provide rich content	30
Let Claude interview you.....	30
Manage your Session.....	30
Course-correct early and often.....	30
Manage context 🤨 <i>Agressively!</i> 😡	31
Use subagents for investigation	31
Rewind with checkpoints	31
Resume conversations.....	31
Automate and scale.....	31
Run non-interactive mode.....	32
Run multiple Claude sessions	32
Fan out across files.....	33
Safe autonomous mode	33
Avoid common failure patterns	33

Start session

Open your terminal in any project directory and start Claude Code:

```
cd /path/to/your/project
claude
```

Essential commands

Here are the most important commands for daily use:

Command	What it does	Example
claude	Start interactive mode	claude
claude "task"	Run a one-time task	claude "fix the build error"
claude -p "query"	Run one-off query, then exit	claude -p "explain this function"
claude -c	Continue most recent conversation in current directory	claude -c
claude -r	Resume a previous conversation	claude -r
claude commit	Create a Git commit	claude commit
/clear	Clear conversation history	/clear
/help	Show available commands	/help
exit or Ctrl+C	Exit Claude Code	exit

Pro tips

<https://code.clause.com/docs/en/best-practices> : Tips and patterns for getting the most out of Claude Code, from configuring your environment to scaling across parallel sessions.

<https://code.clause.com/docs/en/common-workflows> : Step-by-step guides for exploring codebases, fixing bugs, refactoring, testing, and other everyday tasks with Claude Code.

Be specific with your requests

Instead of: “fix the bug” Try: “fix the login bug where users see a blank screen after entering wrong credentials”
Use step-by-step instructions

Break complex tasks into steps:

1. create a new database table for user profiles
2. create an API endpoint to get and update user profiles
3. build a webpage that allows users to see and edit their information

Let Claude explore first

Before making changes, let Claude understand your code:

analyze the database schema

build a dashboard showing products that are most frequently returned by our UK customers

Save time with shortcuts

- Press ? to see all available keyboard shortcuts
- Use Tab for command completion
- Press ↑ for command history
- Type / to see all commands and skills etc

The agentic loop

When you give Claude a task, it works through three phases: **gather context**, **take action**, and **verify results**. These phases blend together. Claude uses tools throughout. The loop adapts to what you ask:

A question about your codebase might only need context gathering. A bug fix cycles through all three phases repeatedly. A refactor might involve extensive verification. Claude decides what each step requires based on *what it learned from the previous step*, chaining dozens of actions together and course-correcting along the way.

Claude Code serves as the **agentic harness** around Claude: it provides the tools, context management, and execution environment that turn a language model into a capable coding agent.

Tools

Tools are what make Claude Code agentic. Without tools, Claude can only respond with text. With tools, Claude can act: read your code, edit files, run commands, search the web, and interact with external services. Each tool use returns information that feeds back into the loop, informing Claude's next decision. The built-in tools generally fall into five categories, each representing a different kind of agency.

Category	What Claude can do
File operations	Read files, edit code, create new files, rename and reorganize
Search	Find files by pattern, search content with regex, explore codebases
Execution	Run shell commands, start servers, run tests, use git
Web	Search the web, fetch documentation, look up error messages
Code intelligence	See type errors and warnings after edits, jump to definitions, find references (requires code intelligence plugins) https://code.claude.com/docs/en/discover-plugins#code-intelligence

These are the primary capabilities. Claude also has tools for spawning subagents, asking you questions, and other orchestration tasks. See <https://code.claude.com/docs/en/settings#tools-available-to-claude> for the complete list.

Work with sessions

Each message, tool use, and result is stored, which enables **rewinding, resuming, and forking** sessions. Before Claude makes code changes, it also snapshots the affected files so you can revert if needed.

Sessions are independent. Each new session starts with a fresh context window, without the conversation history from previous sessions. Claude can persist learnings across sessions using **auto memory**, and you can add your own persistent instructions in **CLAUDE.md**.

Work across branches

Each Claude Code conversation is a session *tied to your current directory*. When you resume, you only see sessions from *that* directory. Claude sees your current branch's files. When you switch branches, Claude sees the new branch's files, but your conversation history stays the same. Claude remembers what you discussed even after switching. Since sessions are tied to directories, you can run parallel Claude sessions by using **git worktrees**, which *create separate directories for individual branches*.

Resume or fork sessions

When you resume a session with Claude --continue or Claude --resume, you pick up where you left off using the same session ID. New messages append to the existing conversation. *Your full conversation history is restored, but **session-scoped permissions are not**. You'll need to **re-approve** those.*

To branch off and try a different approach without affecting the original session, use the --fork-session flag: `claude --continue --fork-session`. Like resume, forked sessions don't inherit session-scoped permissions.

Same session in multiple terminals: If you resume the same session in multiple terminals, both terminals write to the same session file. Messages from both get interleaved, like two people writing in the same notebook. *Nothing corrupts, but the conversation becomes jumbled*. Each terminal only sees its own messages during the session, but if you resume that session later, you'll see everything interleaved. **For parallel work from the same starting point, use --fork-session to give each terminal its own clean session.**

For *parallel work* from the same starting point, use --fork-session to give each terminal its own clean session.

Manage context with skills and subagents

Beyond compaction, you can use other features to control what loads into context. **Skills** load on demand. Claude sees skill descriptions at session start, but the full content only loads when a skill is used. For skills you invoke manually, set `disable-model-invocation: true` to keep descriptions out of context until you need them. **Subagents** get their own fresh context, completely separate from your main conversation. Their work doesn't bloat your context. When done, they return a summary. This isolation is why subagents help with long sessions. See **context costs** for what each feature costs, and **reduce token usage** for tips on managing context.

Checkpoints and permissions

Claude has two safety mechanisms: **Checkpoints** (let you undo file changes) & **Permissions** (control what Claude can do without asking)

Undo changes with checkpoints

Every file edit is reversible. Before Claude edits any file, it snapshots the current contents. If something goes wrong, press Esc twice to rewind to a previous state, or ask Claude to undo.

Work effectively with Claude Code

Ask Claude Code for help: Claude Code can teach you how to use it. Ask questions like “how do I set up hooks?” or “what’s the best way to structure my CLAUDE.md?” and Claude will explain. Built-in commands also guide you through setup:

- `/init` walks you through creating a CLAUDE.md for your project
- `/agents` helps you configure custom subagents
- `/doctor` diagnoses common issues with your installation

It’s a conversation: Claude Code is conversational. You don’t need perfect prompts. Start with what you want, then refine. When the first attempt isn’t right, you don’t start over. You iterate.

Interrupt and steer: You can interrupt Claude at any point. If it’s going down the wrong path, just type your correction and press Enter. Claude will stop what it’s doing and adjust its approach based on your input. You don’t have to wait for it to finish or start over.

Be specific upfront: The more precise your initial prompt, the fewer corrections you’ll need. Reference specific files, mention constraints, and point to example patterns.

Explore before implementing: For complex problems, separate research from coding. Use plan mode (Shift+Tab twice) to analyze the codebase first:

Read src/auth/ and understand how we handle sessions.
Then create a plan for adding OAuth support.

Delegate, don’t dictate: Think of delegating to a capable colleague. Give context and direction, then trust Claude to figure out the details

Extend Claude Code

Extensions plug into different parts of the agentic loop:

- **CLAUDE.md** adds persistent context Claude sees every session
- **Skills** add reusable knowledge and invocable workflows
- **MCP** connects Claude to external services and tools

- **Subagents** run their own loops in isolated context, returning summaries
- **Agent teams** coordinate multiple independent sessions with shared tasks and peer-to-peer messaging
- **Hooks** run outside the loop entirely as deterministic scripts
- **Plugins** and **marketplaces** package and distribute these features

Match features to your goal

Features range from **always-on context** that Claude sees every session, to **on-demand capabilities** you or Claude can invoke, to **background automation** that runs on specific events. The table below shows what's available and when each one makes sense.

Feature	What it does	When to use it	Example
CLAUDE.md	Persistent context loaded every conversation	Project conventions, “always do X” rules	“Use pnpm, not npm. Run tests before committing.”
Skill	Instructions, knowledge, and workflows Claude can use	Reusable content you can load into any context, reference docs, repeatable tasks	/review runs your code review checklist; API docs skill with endpoint patterns
Subagent	Isolated execution context that returns summarized results	Context isolation, parallel tasks, specialized workers	Research task that reads many files but returns only key findings
Agent teams	Coordinate multiple independent Claude Code sessions	Parallel research, new feature development, debugging with competing hypotheses	Spawn reviewers to check security, performance, and tests simultaneously
MCP	Connect to external services	External data or actions	Query your database, post to Slack, control a browser
Hook	Deterministic script that runs on events	Predictable automation, no LLM involved	Run ESLint after every file edit

Plugins are the packaging layer. A plugin bundles skills, hooks, subagents, and MCP servers into a single installable unit. Plugin skills are namespaced (like /my-plugin:review) so multiple plugins can coexist. Use plugins when you want to reuse the same setup across multiple repositories or distribute to others via a **market place** <https://code.claude.com/docs/en/plugin-marketplaces>

Claude.md

Run /init to generate a starter CLAUDE.md file based on your current project structure, then refine over time.

Claude Code reads CLAUDE.md files by walking **up** ↑ the directory  tree  from your *current working directory*, checking **each** directory along the way for CLAUDE.md and CLAUDE.local.md files. This means if you run Claude Code in foo/bar/, it loads instructions from both foo/bar/CLAUDE.md and foo/CLAUDE.md. Claude also discovers CLAUDE.md files in subdirectories **under** ↓ your current working directory .

Use **CLAUDE.md** for instructions *every session needs*: build commands, test conventions, project architecture. For domain knowledge or workflows that are only relevant *sometimes, use skills*.

Use **rules** to keep CLAUDE.md focused. Rules with **paths frontmatter** :

<https://code.claude.com/docs/en/memory#path-specific-rules> only load when Claude works with **matching** files, saving context.

Load from additional directories

The `--add-dir` flag gives Claude access to additional directories outside your main working directory. By default, CLAUDE.md files from these directories are *not* loaded. To also load CLAUDE.md files from additional directories, including CLAUDE.md, `.claude/CLAUDE.md`, and `.claude/rules/*.md`, set the `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` environment variable:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add-dir ../shared-config
```

Organize rules with `.claude/rules/`

For larger projects, you can organize instructions into multiple files using the `.claude/rules/` directory. This keeps instructions modular and easier for teams to maintain. Rules can also be scoped to specific file paths, <https://code.claude.com/docs/en/memory#path-specific-rules>, so they only load into context when Claude works with matching files, reducing noise and saving context space.

Rules load into context *every* session or *when matching files are opened*. For *task-specific instructions* that don't need to be in context all the time, **use skills instead**, which only load when you invoke them or when Claude determines they're relevant to your prompt.

Set up rules

Place markdown files in your project's `.claude/rules/` directory. Each file should cover one topic, with a descriptive filename like `testing.md` or `api-design.md`. All `.md` files are discovered recursively so you can organize rules into subdirectories like `frontend/` or `backend/`:

```
your-project/
├── .claude/
│   ├── CLAUDE.md      # Main project instructions
│   └── rules/
│       ├── code-style.md # Code style guidelines
│       ├── testing.md    # Testing conventions
│       └── security.md   # Security requirements
```

Rules without **paths frontmatter** are loaded at launch with the same priority as `.claude/CLAUDE.md`.

Path-specific rules

Rules can be scoped to specific files using YAML frontmatter with the `paths` field.

```
---
paths:
  - "src/api/**/*.ts"
---
**# API Development Rules**
- All API endpoints must include input validation
```

- Use the standard error response format
- Include OpenAPI documentation comments

Rules without a paths field are loaded unconditionally and apply to all files. Path-scoped rules trigger when Claude reads files matching the pattern, not on every tool use. Use glob patterns in the paths field to match files by extension, directory, or any combination:

Pattern	Matches
<code>**/*.ts</code>	All TypeScript files in any directory
<code>src/**/*.ts</code>	All files under src/ directory
<code>*.md</code>	Markdown files in the project root
<code>src/components/*.tsx</code>	React components in a specific directory

You can specify multiple patterns and use brace expansion to match multiple extensions in one pattern:

```
---
paths:
  - "src/**/*.{ts,tsx}"
  - "lib/**/*.ts"
  - "tests/**/*.test.ts"
---
```

Share rules across projects with symlinks

The `.claude/rules/` directory supports symlinks, so you can maintain a shared set of rules and link them into multiple projects. Symlinks are resolved and loaded normally, and circular symlinks are detected and handled gracefully. This example links both a shared directory and an individual file:

```
ln -s ~/shared-claude-rules .claude/rules/shared
ln -s ~/company-standards/security.md .claude/rules/security.md
```

User-level rules

Personal rules in `~/.claude/rules/` apply to every project on your machine. Use them for preferences that aren't project-specific:

```
~/.claude/rules/
├── preferences.md # Your personal coding preferences
└── workflows.md  # Your preferred workflows
```

User-level rules are loaded before project rules, giving project rules higher priority.

Write effective instructions

CLAUDE.md files are loaded into the context window at the start of every session, consuming tokens alongside your conversation. Because they're context rather than enforced configuration, how you write instructions affects how reliably Claude follows them. Specific, concise, well-structured instructions work best.

Size: Target *under* 200 lines per CLAUDE.md file.

Structure: use markdown headers and bullets to group related instructions. Claude scans structure the same way readers do: organized sections are easier to follow than dense paragraphs.

Consistency: if two rules contradict each other, Claude may pick one arbitrarily. Review your CLAUDE.md files, nested CLAUDE.md files in subdirectories, and **.claude/rules/** <https://code.claude.com/docs/en/memory#organize-rules-with-clauserules> periodically to remove outdated or conflicting instructions. In monorepos, use **claudeMdExcludes** <https://code.claude.com/docs/en/memory#exclude-specific-claude-md-files> to skip CLAUDE.md files from other teams that aren't relevant to your work.

Import additional files

CLAUDE.md files *can import additional files* using `@path/to/import` syntax. Imported files are expanded and loaded into context at launch alongside the CLAUDE.md that references them. Both *relative* and *absolute* paths are allowed. Relative paths resolve relative to the file *containing the import*, **not** the *working directory*. Imported files can recursively import other files, with a maximum depth of five hops. To pull in a README, package.json, and a workflow guide, reference them with `@` syntax anywhere in your CLAUDE.md:

```
See @README for project overview and @package.json for available npm commands for this project.
```

```
# Additional Instructions
- git workflow @docs/git-instructions.md
```

For private per-project preferences that shouldn't be checked into version control, use CLAUDE.local.md: it is *automatically* loaded and added to .gitignore. If you work across multiple git worktrees, CLAUDE.local.md only exists in one. Use a home-directory import instead so **all** worktrees share the same personal instructions:

```
# Individual Preferences
- @~/ .claude/my-project-instructions.md
```

The first time Claude Code encounters external imports in a project, it shows an approval dialog listing the files. If you decline, the imports stay disabled and the dialog does not appear again.

For a more structured approach to organizing instructions, see **.claude/rules/** <https://code.claude.com/docs/en/memory#organize-rules-with-clauserules>.

View and edit with /memory

The `/memory` command lists all CLAUDE.md and rules files loaded in your current session, lets you toggle auto memory on or off, and provides a link to open the auto memory folder. Select any file to open it in your editor. To add instructions to CLAUDE.md instead, ask Claude directly, like “add this to CLAUDE.md,” or edit the file yourself via `/memory`.

Manage CLAUDE.md for large teams

Deploy organization-wide CLAUDE.md

Organizations can deploy a centrally managed CLAUDE.md that applies to all users on a machine. This file cannot be excluded by individual settings.

1 Create the file at the managed policy location

- macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`

- Linux and WSL: /etc/claude-code/CLAUDE.md
- Windows: C:\Program Files\ClaudeCode\CLAUDE.md

2 Deploy with your configuration management system

Use MDM, Group Policy, Ansible, or similar tools to distribute the file across developer machines. See **managed settings** <https://code.claude.com/docs/en/permissions#managed-settings> for *other* organization-wide configuration options.

Exclude specific CLAUDE.md files

The `claudeMdExcludes` setting lets you skip specific files by path or glob pattern. This example excludes a top-level CLAUDE.md and a rules directory from a parent folder. Add it to `.claude/settings.local.json` so the exclusion stays local to your machine:

```
{
  "claudeMdExcludes": [
    "**/monorepo/CLAUDE.md",
    "/home/user/monorepo/other-team/.claude/rules/**"
  ]
}
```

Patterns are matched against absolute file paths using glob syntax. You can configure `claudeMdExcludes` at any **settings layer** <https://code.claude.com/docs/en/settings#settings-files>: user, project, local, or managed policy. Arrays merge across layers. Managed policy CLAUDE.md files *cannot* be excluded. This ensures organization-wide instructions ***always apply regardless of individual settings***.

✅ Include	❌ Exclude
Bash commands Claude can't guess	Anything Claude can figure out by reading code
Code style rules that differ from defaults	Standard language conventions Claude already knows
Testing instructions and preferred test runners	Detailed API documentation (link to docs instead)
Repository etiquette (branch naming, PR conventions)	Information that changes frequently
Architectural decisions specific to your project	Long explanations or tutorials
Developer environment quirks (required env vars)	File-by-file descriptions of the codebase
Common gotchas or non-obvious behaviors	Self-evident practices like "write clean code"

1. If Claude keeps doing something you don't want despite having a rule against it, the file is probably too long and the rule is getting lost.
2. If Claude asks you questions that are answered in CLAUDE.md, the phrasing might be ambiguous.
3. Treat CLAUDE.md like code: review it when things go wrong, prune it regularly, and test changes by observing whether Claude's behavior actually shifts.
4. You can tune instructions by adding emphasis (e.g., "IMPORTANT" or "YOU MUST") to improve adherence. Check CLAUDE.md into git so your team can contribute.

CLAUDE.md files can import additional files using @path/to/import syntax:

CLAUDE.md

See @README.md for project overview and @package.json for available npm commands.

Additional Instructions

- Git workflow: @docs/git-instructions.md
- Personal overrides: @~/ .claude/my-project-instructions.md

Claude.md Locations

You can place CLAUDE.md files in several locations:

- **Home folder** (~/.claude/CLAUDE.md): applies to all Claude sessions
- **Project root** (./CLAUDE.md): check into git to share with your team, or name it CLAUDE.local.md and .gitignore it
- **Parent directories**: useful for monorepos where both root/CLAUDE.md and root/foo/CLAUDE.md are pulled in automatically

Auto memory

Claude Code has two complementary memory systems. Both are loaded at the start of every conversation. Claude treats them as *context*, **not enforced configuration**. The more specific and concise your instructions, the more consistently Claude follows them.

	CLAUDE.md files	Auto memory
Who writes it	You	Claude
What it contains	Instructions and rules	Learnings and patterns
Scope	Project, user, or org	Per working tree
Loaded into	Every session	Every session (first 200 lines)
Use for	Coding standards, workflows, project architecture	Build commands, debugging insights, preferences Claude discovers

Use CLAUDE.md files when you want to guide Claude's behavior. Auto memory lets Claude learn from your corrections without manual effort.

Subagents can also maintain their own auto memory: <https://code.claude.com/docs/en/sub-agents#enable-persistent-memory>

The memory field gives the subagent a *persistent directory* that *survives across conversations*. The subagent uses this directory to build up knowledge over time, such as *codebase patterns*, *debugging insights*, and *architectural decisions*.

```
---
name: code-reviewer
description: Reviews code for quality and best practices
memory: user
---
```

You are a code reviewer. As you review code, update your agent memory with patterns, conventions, and recurring issues you discover.

Auto memory lets Claude accumulate knowledge across sessions without you writing anything. Claude saves notes for itself as it works: build commands, debugging insights, architecture notes, code style preferences, and workflow habits. Claude doesn't save something every session. It decides what's worth remembering based on whether the information would be useful in a future conversation.

Enable or disable auto memory

Auto memory is on by default. To toggle it, open `/memory` in a session and use the auto memory toggle, or set `autoMemoryEnabled` in your project settings:

```
{
  "autoMemoryEnabled": false
}
```

To disable auto memory via environment variable, set `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`

Storage Location

Each project gets its own *memory directory* at `~/.claude/projects/<project>/memory/`. The path is derived from the *git repository*, so **all** worktrees and subdirectories within the *same repo* share one auto memory directory. *Outside* a git repo, the project root is used instead. The directory contains a `MEMORY.md` entrypoint and optional topic files:

```
~/.claude/projects/<project>/memory/
├── MEMORY.md      # Concise index, loaded into every session
├── debugging.md   # Detailed notes on debugging patterns
├── api-conventions.md # API design decisions
└── ...           # Any other topic files Claude creates
```

`MEMORY.md` acts as an *index* of the memory directory. Claude reads and writes files in this directory throughout your session, using `MEMORY.md` to *keep track* of what's stored where. Auto memory is *machine-local*. All worktrees and subdirectories *within the same git repository* share one auto memory directory. Files are **not** shared across machines or cloud environments.

How it works

The first 200 lines of `MEMORY.md` are loaded at the start of every conversation.

Configure permissions

Use `/permissions` to allowlist safe commands or `/sandbox` for OS-level isolation.

- **Permission allowlists:** permit specific tools you know are safe (like `npm run lint` or `git commit`)
- **Sandboxing:** enable OS-level isolation that restricts filesystem and network access, allowing Claude to work more freely within defined boundaries

CLI Tools

Tell Claude Code to use CLI tools like `gh`, `aws`, `gcloud`, and `sentry-cli` when interacting with external services.

CLI tools are the *most context-efficient* way to interact with external services. If you use GitHub, install the gh CLI. Claude *knows* how to use it for creating issues, opening pull requests, and reading comments. Without gh, Claude can still use the GitHub API, but unauthenticated requests *often hit rate limits*. Claude is also *effective at learning CLI tools it doesn't already know*. Try prompts like Use foo-cli-tool --help to learn about foo tool, then use it to solve A, B, C.

Skills

<https://code.claude.com/docs/en/skills>

A skill is a *markdown file* containing *knowledge*, *workflows*, or *instructions*. You can invoke skills with a command like /deploy, or Claude can load them automatically when relevant. Skills can run in your current conversation or in an isolated context via subagents.

Skills can be reference or action. Reference skills provide *knowledge* Claude uses throughout your session (like your API style guide). Action skills *tell Claude to do something specific* (like /deploy that runs your deployment workflow).

Use skills for content Claude only needs *sometimes*, like API documentation or a deployment checklist you trigger with /<name>

When: Depends on the skill's configuration. By default, descriptions load at session start and full content loads when used. For *user-only* skills (i.e., disable-model-invocation: true), nothing loads *until you invoke them*.

What loads: For *model-invocable* skills, Claude sees names and descriptions in *every* request. When you invoke a skill with / or Claude loads it automatically, the full content loads into your conversation.

How Claude chooses skills: Claude *matches* your *task* against *skill descriptions* to decide which are relevant. If descriptions are *vague* or *overlap*, Claude may load the wrong skill or miss one that would help. To tell Claude to use a specific skill, invoke it with /. Skills with disable-model-invocation: true are invisible to Claude until you invoke them.

Context cost: Low until used. User-only skills have zero cost until invoked.

In subagents: Skills work differently in subagents. Instead of on-demand loading, *skills passed to a subagent are fully preloaded into its context at launch*. Subagents *don't* inherit skills from the main session; *you must specify them explicitly*.

Use disable-model-invocation: true for skills with *side effects*. This saves context and ensures *only you* trigger them.

.claude/skills/api-conventions/SKILL.md

```
---
name: api-conventions
description: REST API design conventions for our services
---

# API Conventions
- Use kebab-case for URL paths
- Use camelCase for JSON properties
- Always include pagination for list endpoints
- Version APIs in the URL path (/v1/, /v2/)
```

Skills can also define repeatable workflows you invoke directly: `.claude/skills/fix-issue/SKILL.md`

```
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---

Analyze and fix the GitHub issue: $ARGUMENTS.

1. Use `gh issue view` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes to fix the issue
5. Write and run tests to verify the fix
6. Ensure code passes linting and type checking
7. Create a descriptive commit message
8. Push and create a PR
```

Run `/fix-issue 1234` to invoke it. Use `disable-model-invocation: true` for workflows with side effects that you want to trigger manually.

MCP

Connect MCP servers

Run `claude mcp add` to connect external tools like Notion, Figma, or your database.

With **MCP servers** <https://code.claude.com/docs/en/mcp>, you can ask Claude to implement features from issue trackers, query databases, analyze monitoring data, integrate designs from Figma, and automate workflows.

Subagents

Use subagents for work that doesn't need your full conversation context. Their isolation prevents bloating your main session.

Use a subagent when you need *context isolation* or when your *context window is getting full*. The subagent might read dozens of files or run extensive searches, but your **main** conversation only *receives a summary*. Since subagent work doesn't consume your main context, this is also useful *when you don't need the intermediate work to remain visible*.

When: On demand, when you or Claude spawns one for a task.

What loads: Fresh, isolated context containing:

- The system prompt (shared with parent for cache efficiency)
- Full content of skills listed in the agent's skills: field
- CLAUDE.md and git status (inherited from parent)
- Whatever context the lead agent passes in the prompt

Context cost: Isolated from main session. Subagents don't inherit your conversation history or invoked skills.

```
---
name: api-developer
description: Implements REST endpoints following team conventions. Use PROACTIVELY for new API work.
tools: Read, Write, Edit, Bash
model: inherit
context: fork
skills:
  - api-conventions
  - error-handling
  - openapi-spec
disable-model-invocation: true
---
```

You are an API developer. Follow the preloaded conventions strictly...

Deploy the application:

1. Run the test suite
2. Build the application
3. Push to the deployment target

Custom subagents can have *their own instructions* and *can preload skills* (skills: field). A skill can run in isolated context using context: fork. See <https://code.claude.com/docs/en/skills> for details.

Transition point: If you're running *parallel subagents* but hitting context limits, or if your subagents need to communicate with each other, *agent teams* are the natural next step.

Use specialized subagents

Suppose you want to use specialized AI subagents to handle specific tasks more effectively.

1 View available subagents: This shows all available subagents and lets you create new ones.

/agents

2 Use subagents automatically

Claude Code automatically delegates appropriate tasks to specialized subagents:

review my recent code changes for security issues
run all tests and fix any failures

3 Explicitly request specific subagents

use the code-reviewer subagent to check the auth module
have the debugger subagent investigate why users can't log in

4 Create custom subagents for your workflow

/agents

Then select "Create New subagent" and follow the prompts to define:

- A unique identifier that describes the subagent's purpose (for example, code-reviewer, api-designer).
- When Claude should use this agent
- Which tools it can access
- A system prompt describing the agent's role and behavior

Tips:

- Create project-specific subagents in **.claude/agents/** for team sharing
- Use descriptive description fields to enable *automatic delegation*
- Limit tool access to what each subagent *actually needs*

Check the **subagents documentation** <https://code.claude.com/docs/en/sub-agents> for detailed examples

Subagents run in their own context with their own set of allowed tools. They're useful for tasks that read many files or need specialized focus without cluttering your main conversation.

.claude/agents/security-reviewer.md

```
---
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: opus

---
You are a senior security engineer. Review code for:
- Injection vulnerabilities (SQL, XSS, command injection)
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure data handling
```

Provide specific line references and suggested fixes.

Tell Claude to use subagents explicitly: *"Use a subagent to review this code for security issues."*

Agent teams

Use an agent team when teammates need to

1. share findings
2. challenge each other
3. coordinate independently.

Agent teams are best for research with competing hypotheses, parallel code review, and new feature development where each teammate owns a separate piece.

Agent teams are experimental and disabled by default.

to your settings.json or environment. Agent teams have known limitations around session resumption, task coordination, and shutdown behavior.

See [agent teams](#) for setup and current limitations.

Hooks

When: On trigger. Hooks fire at specific lifecycle events like tool execution, session boundaries, prompt submission, permission requests, and compaction. See [Hooks](#) for the full list.

What loads: Nothing by default. Hooks run as external scripts.

Context cost: Zero, unless the hook returns output that gets added as messages to your conversation.

Hooks are ideal for side effects (linting, logging) that don't need to affect Claude's context.

Each feature has its own guide with setup instructions, examples, and configuration options.

Set up hooks

<https://code.claude.com/docs/en/hooks-guide>

<https://code.claude.com/docs/en/best-practices#set-up-hooks>

Use hooks for actions that must happen every time with zero exceptions.

Hooks run scripts automatically at specific points in Claude's workflow. Unlike CLAUDE.md instructions which are advisory, hooks are deterministic and guarantee the action happens. Claude can write hooks for you. Try prompts like *"Write a hook that runs eslint after every file edit"* or *"Write a hook that blocks writes to the migrations folder."* Run `/hooks` for interactive configuration, or edit `.claude/settings.json` directly.

Plugins

<https://code.claude.com/docs/en/plugins>)

Run `/plugin` to browse the marketplace. Plugins add skills, tools, and integrations without configuration.

Plugins bundle skills, hooks, subagents, and MCP servers into a single installable unit from the community and Anthropic. If you work with a typed language, install a **code intelligence plugin** <https://code.claude.com/docs/en/discover-plugins#code-intelligence> to give Claude precise symbol navigation and automatic error detection after edits. For guidance on choosing between skills, subagents, hooks, and MCP, see **Extend Claude Code** <https://code.claude.com/docs/en/features-overview#match-features-to-your-goal>

Understand how features layer

Features can be defined at multiple levels: user-wide, per-project, via plugins, or through managed policies. You can also nest CLAUDE.md files in subdirectories or place skills in specific packages of a monorepo. When the same feature exists at multiple levels, here's how they layer:

- **CLAUDE.md files** are *additive*: all levels contribute content to Claude's context simultaneously. Files from your working directory and above load at launch; subdirectories load as you work in them. When instructions conflict, Claude uses judgment to reconcile them, with more specific

instructions typically taking precedence. See **how CLAUDE.md files load** <https://code.claude.com/docs/en/memory#how-claude-md-files-load>.

- **Skills and subagents** override by name: when the same name exists at multiple levels, one definition wins based on *priority* (managed > user > project for skills; managed > CLI flag > project > user > plugin for subagents). Plugin skills are **namespaced** to avoid conflicts. See **skill discovery** and **subagent scope**.
- **MCP servers** override by name: local > project > user. See **MCP scope**.
- **Hooks** merge: all registered hooks fire for their matching events regardless of source. See **hooks**.

Understand context costs

Every feature you add consumes some of Claude's context. Too much can fill up your context window, but it can also add noise that makes Claude less effective; skills may not trigger correctly, or Claude may lose track of your conventions. Understanding these trade-offs helps you build an effective setup.

Context cost by feature

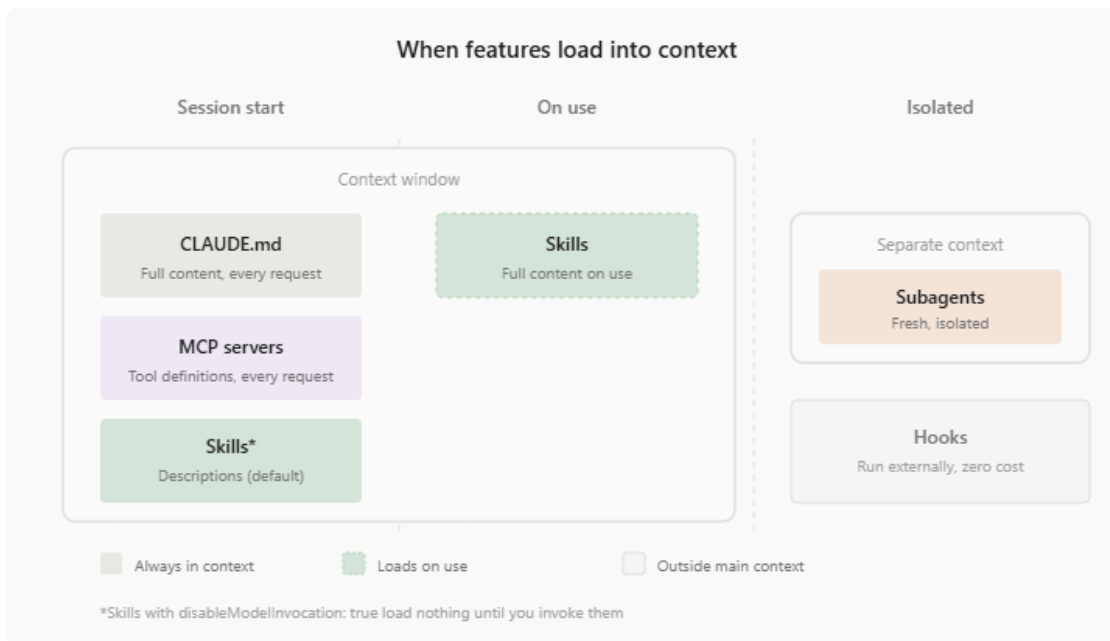
Each feature has a different loading strategy and context cost:

Feature	When it loads	What loads	Context cost
CLAUDE.md	Session start	Full content	Every request
Skills	Session start + when used	Descriptions at start, full content when used	Low (descriptions every request)*
MCP servers	Session start	All tool definitions and schemas	Every request
Subagents	When spawned	Fresh context with specified skills	Isolated from main session
Hooks	On trigger	Nothing (runs externally)	Zero, unless hook returns additional context

*By default, skill descriptions load at session start so Claude can decide when to use them. Set `disable-model-invocation: true` in a skill's frontmatter to hide it from Claude entirely until you invoke it manually. This reduces context cost to zero for skills you only trigger yourself.

Understand how features load

Each feature loads at different points in your session. The tabs below explain when each one loads and what goes into context.



Combine features

Each extension solves a different problem:

- **CLAUDE.md** handles always-on context,
- **skills handle** on-demand knowledge and workflows,
- **MCP** handles external connections,
- **subagents** handle isolation, and
- **hooks** handle automation.

Real setups combine them based on your workflow. For example, you might use CLAUDE.md for project conventions, a skill for your deployment workflow, MCP to connect to your database, and a hook to run linting after every edit. Each feature handles what it’s best at.

Pattern	How it works	Example
Skill + MCP	MCP provides the connection; a skill teaches Claude how to use it well	MCP connects to your database, a skill documents your schema and query patterns
Skill + Subagent	A skill spawns subagents for parallel work	/review skill kicks off security, performance, and style subagents that work in isolated context
CLAUDE.md + Skills	CLAUDE.md holds always-on rules; skills hold reference material loaded on demand	CLAUDE.md says “follow our API conventions,” a skill contains the full API style guide
Hook + MCP	A hook triggers external actions through MCP	Post-edit hook sends a Slack notification when Claude modifies critical files

Plan Mode

You can switch into Plan Mode during a session using **Shift+Tab** to cycle through permission modes.

When to use Plan Mode

- **Multi-step implementation:** When your feature requires making edits to many files
- **Code exploration:** When you want to research the codebase thoroughly before changing anything
- **Interactive development:** When you want to iterate on the direction with Claude

Start a new session in Plan Mode

```
claude --permission-mode plan
```

Run “headless” queries in Plan Mode: You can also run a query in Plan Mode directly with -p (that is, in “headless mode”):

```
claude --permission-mode plan -p "Analyze the authentication system and suggest improvements"
```

💡 Press Ctrl+Alt+G to open the plan in your default text editor, where you can edit it directly *before* Claude proceeds.

Explore first, then plan, then code

1 Explore

Enter Plan Mode. Claude reads files and answers questions without making changes.

```
# claude (Plan Mode)
read /src/auth and understand how we handle sessions and login. Also look at how we manage
environment variables for secrets.
```

2 Plan: Ask Claude to create a detailed implementation plan.

```
# claude (Plan Mode)
I want to add Google OAuth. What files need to change?
What's the session flow? Create a plan.
```

Press Ctrl+ALT+G to open the plan in your text editor for direct editing before Claude proceeds.

3 Implement: Switch back to Normal Mode and let Claude code, verifying against its plan.

```
# claude (Normal Mode)
implement the OAuth flow from your plan. write tests for the callback handler, run the
test suite and fix any failures.
```

4 Commit: Ask Claude to commit with a descriptive message and create a PR.

```
# claude (Normal Mode)
commit with a descriptive message and open a PR
```

💡 Plan Mode is useful, but also *adds overhead*. For tasks where the scope is clear and the fix is small (like fixing a typo, adding a log line, or renaming a variable) ask Claude to do it *directly*. Planning is most useful *when you're uncertain about the approach*, when the change modifies multiple files, or when you're unfamiliar with the code being modified. ***If you could describe the diff in one sentence, skip the plan.***

Configure Plan Mode as default

```
// .claude/settings.json
{
  "permissions": {
    "defaultMode": "plan"
  }
}
```

Extended thinking

<https://platform.claude.com/docs/en/build-with-claude/extended-thinking>

Extended thinking controls *how much internal reasoning* Claude performs *before* responding. More thinking provides more space to explore solutions, analyze edge cases, and self-correct mistakes. E.T gives Claude *enhanced reasoning* capabilities for complex tasks, while providing *varying levels of transparency* into its step-by-step thought process before it delivers its final answer.

💡 On claude-fable-5 and claude-mythos-5, extended thinking is ***always*** enabled and *cannot* be disabled. Manual extended thinking (thinking: {type: "enabled", budget_tokens: N}) is not supported; use [adaptive thinking](#) instead. Adaptive thinking is always on, and thinking: {type: "disabled"} returns an error.

💡 For Claude Opus 4.8 and Claude Opus 4.7, set thinking: {type: "adaptive"} to enable [adaptive thinking](#) and use the [effort parameter](#) to control thinking depth. On both models, manual extended thinking (thinking: {type: "enabled", budget_tokens: N}) is not supported and returns a 400 error. With adaptive thinking, the model decides when and how much to think based on each request, so it triggers thinking only as needed. For Claude Opus 4.6 and Claude Sonnet 4.6, adaptive thinking is also recommended; the manual configuration is still functional on these models but is deprecated and will be removed in a future model release.

Adaptive thinking

<https://platform.claude.com/docs/en/build-with-claude/adaptive-thinking>

Adaptive thinking is the recommended way to use [extended thinking](#) with Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, and Claude Sonnet 4.6. On Claude Fable 5 and [Claude Mythos 5](#), thinking is always enabled and cannot be disabled; adaptive thinking is the only thinking mode. On [Claude Mythos Preview](#), adaptive thinking is the default mode and auto-applies whenever thinking is unset. Instead of manually setting a thinking token budget, adaptive thinking lets Claude dynamically determine when and how much to use extended thinking based on the complexity of each request. On Claude Opus 4.8 and Claude Opus 4.7, adaptive thinking is the **only** supported thinking mode; manual thinking: {type: "enabled", budget_tokens: N} is no longer accepted.

Adaptive thinking can drive better performance than extended thinking with a fixed budget_tokens for many workloads, especially bimodal tasks and long-horizon agentic workflows. No beta header is required.

If your workload requires predictable latency or precise control over thinking costs, extended thinking with `budget_tokens` is still functional on Claude Opus 4.6 and Claude Sonnet 4.6 but is deprecated and no longer recommended. See the warning below.

Adaptive thinking with the effort parameter

You can combine adaptive thinking with the **effort parameter** <https://platform.claude.com/docs/en/build-with-claude/effort> to guide how much thinking Claude does. The effort level acts as soft guidance for Claude's thinking allocation:

Effort level	Thinking behavior
max	Claude always thinks with no constraints on thinking depth. Available on Claude Fable 5, Claude Mythos 5, Claude Mythos Preview, Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, and Claude Sonnet 4.6.
xhigh	Claude always thinks deeply with extended exploration. Available on Claude Fable 5, Claude Mythos 5, Claude Opus 4.8, and Claude Opus 4.7.
high (default)	Claude almost always thinks. Provides deep reasoning on complex tasks.
medium	Claude uses moderate thinking. May skip thinking for very simple queries.
low	Claude minimizes thinking. Skips thinking for simple tasks where speed matters most.

```
# Python
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=16000,
    thinking={"type": "adaptive"},
    output_config={"effort": "medium"},
    messages=[{"role": "user", "content": "What is the capital of France?"}],
)

print(response.content[0].text)
```

Session

Resume previous conversations

When starting Claude Code, you can resume a previous session:

- `claude --continue` continues the most recent conversation in the current directory
- `claude --resume` opens a conversation picker or resumes by name
- `claude --from-pr 123` resumes sessions linked to a specific pull request

From inside an active session, use `/resume` to switch to a different conversation. Sessions are stored per project directory. The `/resume` picker shows sessions from the same git repository, including worktrees.

Name your sessions

Give sessions *descriptive* names to find them later. This is a best practice when working on multiple tasks or features.

1 Name the current session: Use `/rename` during a session to give it a memorable name: `/rename auth-refactor` You can also rename any session from the picker: run `/resume`, navigate to a session, and press `R`.

2 Resume by name later: From the command line: `claude --resume auth-refactor` Or from inside an active session: `/resume auth-refactor`

Use the session picker

<https://code.claude.com/docs/en/common-workflows#use-the-session-picker>

The `/resume` command (or `claude --resume` without arguments) opens an interactive session picker with these features:**Keyboard shortcuts in the picker:**

Shortcut	Action
↑ / ↓	Navigate between sessions
→ / ←	Expand or collapse grouped sessions
Enter	Select and resume the highlighted session
P	Preview the session content
R	Rename the highlighted session
/	Search to filter sessions
A	Toggle between current directory and all projects
B	Filter to sessions from your current git branch
Esc	Exit the picker or search mode

Session organization:

The picker displays sessions with helpful metadata:

- Session name or initial prompt
- Time elapsed since last activity
- Message count
- Git branch (if applicable)

Forked sessions (created with `/rewind` or `--fork-session`) are grouped together under their root session, making it easier to find related conversations.

Tips:

- **Name sessions early:** Use `/rename` when starting work on a distinct task—it's much easier to find "payment-integration" than "explain this function" later
- Use `--continue` for *quick access to your most recent conversation* in the current directory
- Use `--resume session-name` when you know which session you need
- Use `--resume` (without a name) when you need to *browse and select*
- For scripts, use `claude --continue --print "prompt"` to resume in non-interactive mode
- Press P in the picker to preview a session before resuming it
- The resumed conversation starts with the same model and configuration as the original

Run parallel Claude Code sessions with Git worktrees

When working on *multiple tasks at once*, you need each Claude session to have *its own copy of the codebase* so changes don't collide. Git worktrees solve this by creating separate working directories that each have their *own files and branch*, while sharing the same repository history and remote connections. This means you can have Claude working on a feature in one worktree while fixing a bug in another, without either session interfering with the other.

Use the `--worktree (-w)` flag to create an isolated worktree and start Claude in it. The value you pass becomes the worktree directory name and branch name:

```
# Start Claude in a worktree named "feature-auth"
# Creates .claude/worktrees/feature-auth/ with a new branch
claude --worktree feature-auth

# Start another session in a separate worktree
claude --worktree bugfix-123
```

If you omit the name, Claude generates a random one automatically:

```
# Auto-generates a name like "bright-running-fox"
claude --worktree
```

Worktrees are created at `<repo>/ .claude/worktrees/<name>` and branch from the default remote branch. The worktree branch is named `worktree-`. You can also ask Claude to “work in a worktree” or “start a worktree” during a session, and it will create one automatically.

Subagent worktrees

Subagents can also use worktree isolation to *work in parallel without conflicts*. Ask Claude to “use worktrees for your agents” or configure it in a **custom subagent** by adding `isolation: worktree` to the agent's frontmatter. Each subagent gets its own worktree that is automatically cleaned up when the subagent finishes without changes.

Worktree cleanup

When you exit a worktree session, Claude handles cleanup based on whether you made changes:

- **No changes:** the worktree and its branch are removed automatically

Changes or commits exist: Claude prompts you to keep or remove the worktree.

To clean up worktrees outside of a Claude session, use [manual worktree management](#).

Add `.claude/worktrees/` to your `.gitignore` to prevent worktree contents from appearing as untracked files in your main repository.

Manage worktrees manually

For more control over worktree location and branch configuration, create worktrees with Git directly. This is useful when you need to check out a specific existing branch or place the worktree outside the repository.

Create a worktree with a new branch

```
git worktree add ../project-feature-a -b feature-a

# Create a worktree with an existing branch
git worktree add ../project-bugfix bugfix-123

# Start Claude in the worktree
cd ../project-feature-a && claude

# Clean up when done
git worktree list
git worktree remove ../project-feature-a
```

Learn more in the **official Git worktree documentation** <https://git-scm.com/docs/git-worktree>

Remember to initialize your development environment in each new worktree according to your project's setup. Depending on your stack, this might include running dependency installation (npm install, yarn), setting up virtual environments, or following your project's standard setup process.

Common Workflows

Working with Tests

When asking for tests, be specific about *what behavior you want to verify*. Claude examines your existing test files to match the style, frameworks, and assertion patterns already in use. For comprehensive coverage, ask Claude to identify edge cases you might have missed.

Create pull requests

You can create pull requests by asking Claude directly ("create a pr for my changes") or by using the `/commit-push-pr` skill, which commits, pushes, and opens a PR in one step.

```
/commit-push-pr
```

When you create a PR using `gh pr create`, the *session* is automatically linked to that *PR*. You can resume it later with `claude --from-pr <number>`.

Reference files and directories

Use `@` to *quickly* include files or directories *without waiting for Claude to read them*.

1 Reference a single file

Explain the logic in @src/utils/auth.js

This includes the full content of the file in the conversation.

2 Reference a directory : This provides a directory listing with file information.

What's the structure of @src/components?

3 Reference MCP resources: This fetches data from connected MCP servers using the format @server:resource.

Show me the data from @github:repos/owner/repo/issues

See **MCP resources** <https://code.claude.com/docs/en/mcp#use-mcp-resources> for details.

Tips:

- File paths can be relative or absolute
- @ file references add CLAUDE.md in the file's directory and parent directories to context
- Directory references show file listings, not contents
- You can reference multiple files in a single message (for example, "@file1.js and @file2.js")

Pipe in, pipe out

Suppose you want to pipe data into Claude, and get back data in a structured format.

Pipe data through Claude:

```
cat build-error.txt | claude -p 'concisely explain the root cause of this build error' > output.txt
```

Consider using output-format for structured output

1 Use text format (default)

```
cat data.txt | claude -p 'summarize this data' --output-format text > summary.txt
```

This outputs just Claude's plain text response (default behavior).

2 Use JSON format

```
cat code.py | claude -p 'analyze this code for bugs' --output-format json > analysis.json
```

This outputs a JSON array of messages with metadata including cost and duration.

3 Use streaming JSON format

```
cat log.txt | claude -p 'parse this log file for errors' --output-format stream-json
```

This outputs a series of JSON objects in real-time as Claude processes the request. Each message is a valid JSON object, but the entire output is not valid JSON if concatenated.

Tips:

- Use `--output-format text` for simple integrations where you just need Claude's response
- Use `--output-format json` when you need the full conversation log
- Use `--output-format stream-json` for real-time output of each conversation turn

Best Practices

Give cc a way to verify its work

Strategy	Before	After
Provide verification criteria	<i>"implement a function that validates email addresses"</i>	<i>"write a validateEmail function. example test cases: *user@example.com* is true, invalid is false, *user@.com* is false. run the tests after implementing"</i>
Verify UI changes visually	<i>"make the dashboard look better"</i>	<i>"[paste screenshot] implement this design. take a screenshot of the result and compare it to the original. list differences and fix them"</i>
Address root causes, not symptoms	<i>"the build is failing"</i>	<i>"the build fails with this error: [paste error]. fix it and verify the build succeeds. address the root cause, don't suppress the error"</i>

Provide specific context in your prompts

Claude can infer intent, but it can't read your mind. Reference specific files, mention constraints, and point to example patterns.

Strategy	Before	After
Scope the task. Specify which file, what scenario, and testing preferences.	<i>"add tests for foo.py"</i>	<i>"write a test for foo.py covering the edge case where the user is logged out. avoid mocks."</i>
Point to sources. Direct Claude to the source that can answer a question.	<i>"why does ExecutionFactory have such a weird api?"</i>	<i>"look through ExecutionFactory's git history and summarize how its api came to be"</i>
Reference existing patterns. Point Claude to patterns in your codebase.	<i>"add a calendar widget"</i>	<i>"look at how existing widgets are implemented on the home page to understand the patterns. HotDogWidget.php is a good example. follow the pattern to implement a new calendar widget that lets the user select a month and paginate forwards/backwards to pick a year. build from scratch without libraries other than the ones already used in the codebase."</i>
Describe the symptom. Provide the symptom, the likely location, and what "fixed" looks like.	<i>"fix the login bug"</i>	<i>"users report that login fails after session timeout. check the auth flow in src/auth/, especially token refresh. write a failing test that reproduces the issue, then fix it"</i>

Vague prompts can be useful when you're exploring and can afford to course-correct. A prompt like "what would you improve in this file?" can surface things you wouldn't have thought to ask about.

Provide rich content

- **Reference files with @** instead of describing where code lives. Claude reads the file before responding.
- **Paste images directly.** Copy/paste or drag and drop images into the prompt.
- **Give URLs** for documentation and API references. Use `/permissions` to allowlist frequently-used domains.
- **Pipe in data** by running `cat error.log | Claude` to send file contents directly.

Let Claude fetch what it needs. Tell Claude to pull context itself using Bash commands, MCP tools, or by reading files.

Let Claude interview you

For larger features, have Claude interview you first. Start with a minimal prompt and ask Claude to interview you using the AskUserQuestion tool.

Claude asks about things you might not have considered yet, including technical implementation, UI/UX, edge cases, and tradeoffs.

I want to build [brief description]. Interview me in detail using the AskUserQuestion tool.

Ask about technical implementation, UI/UX, edge cases, concerns, and tradeoffs. Don't ask obvious questions, dig into the hard parts I might not have considered.

Keep interviewing until we've covered everything, then write a complete spec to SPEC.md

Once the spec is complete, start a fresh session to execute it. The new session has clean context focused entirely on implementation, and you have a written spec to reference.

Manage your Session

Course-correct early and often

Correct Claude as soon as you notice it going off track.

- **Esc:** stop Claude mid-action with the Esc key. Context is preserved, so you can redirect.
- **Esc + Esc or /rewind:** press Esc twice or run `/rewind` to open the rewind menu and restore previous conversation and code state, or summarize from a selected message.

If you've corrected Claude more than *twice on the same issue* in one session, the context is cluttered with failed approaches. Run `/clear` and start fresh with a more specific prompt that incorporates what you learned. A clean session with a better prompt almost always outperforms a long session with accumulated corrections.

Manage context 🧐 Agressively! 🧐

Claude Code automatically compacts conversation history when you approach context limits.

During long sessions, Claude's context window can fill with irrelevant conversation, file contents, and commands.

- To compact only part of the conversation, use Esc + Esc or /rewind, select a message checkpoint, and choose **Summarize from here**. This condenses messages from that point forward while keeping earlier context intact.
- Customize compaction behavior in CLAUDE.md with instructions like "When compacting, always preserve the full list of modified files and any test commands" to ensure critical context survives summarization

Use subagents for investigation

Delegate research with "use subagents to investigate X". They explore in a separate context, keeping your main conversation clean for implementation.

Since context is your fundamental constraint, *subagents are one of the most powerful tools available*. Subagents run in separate context windows and *report back summaries*

Rewind with checkpoints

Every action Claude makes creates a checkpoint.

Double-tap Escape or run /rewind to open the rewind menu. You can restore conversation only, restore code only, restore both, or summarize from a selected message. See **Checkpointing** for details <https://code.claude.com/docs/en/checkpointing>. Instead of carefully planning every move, you can tell Claude to try something risky. If it doesn't work, rewind and try a different approach. Checkpoints persist across sessions, so you can close your terminal and still rewind later.

Resume conversations

Run `claude --continue` to pick up where you left off, or `--resume` to choose from recent sessions.

Claude Code saves conversations locally. When a task spans multiple sessions, you don't have to re-explain the context:

```
claude --continue # Resume the most recent conversation
claude --resume   # Select from recent conversations
```

Use /rename to give sessions descriptive names like "oauth-migration" or "debugging-memory-leak" so you can find them later. Treat sessions like branches: different workstreams can have separate, persistent contexts.

Automate and scale

Once you're effective with one Claude, multiply your output with parallel sessions, non-interactive mode, and fan-out patterns.

Run non-interactive mode

Use `claude -p "prompt"` in [CI](#), [pre-commit hooks](#), or [scripts](#). Add `--output-format stream-json` for streaming JSON output.

With `claude -p "your prompt"`, you can run Claude *non-interactively*, ***without a session***. Non-interactive mode is how you integrate Claude into *CI pipelines*, *pre-commit hooks*, or any *automated workflow*.

```
# One-off queries
claude -p "Explain what this project does"

# Structured output for scripts
claude -p "List all API endpoints" --output-format json

# Streaming for real-time processing
claude -p "Analyze this log file" --output-format stream-json
```

Run multiple Claude sessions

Run multiple Claude sessions in parallel to speed up development, run isolated experiments, or start complex workflows.

There are three main ways to run parallel sessions:

- **Claude Code desktop app:** Manage multiple local sessions visually. Each session gets its own isolated worktree. <https://code.claude.com/docs/en/desktop#work-in-parallel-with-sessions>
- **Claude Code on the web:** Run on Anthropic's secure cloud infrastructure in isolated VMs. <https://code.claude.com/docs/en/claude-code-on-the-web>
- **Agent teams:** Automated coordination of multiple sessions with shared tasks, messaging, and a team lead. <https://code.claude.com/docs/en/agent-teams>

A fresh context improves code review since Claude won't be biased toward code it just wrote. For example, use a Writer/Reviewer pattern:

Session A (Writer)	Session B (Reviewer)
Implement a rate limiter for our API endpoints	
	Review the rate limiter implementation in <code>@src/middleware/rateLimiter.ts</code> . Look for edge cases, race conditions, and consistency with our existing middleware patterns.
Here's the review feedback: [Session B output]. Address these issues.	

Fan out across files

Loop through tasks calling `claude -p` for each. Use `--allowedTools` to scope permissions for batch operations.

For large migrations or analyses, you can distribute work across many parallel Claude invocations:

1 Generate a task list

Have Claude list all files that need migrating (e.g., list all 2,000 Python files that need migrating)

2 Write a script to loop through the list

```
for file in $(cat files.txt); do
  claude -p "Migrate $file from React to Vue. Return OK or FAIL." \
    --allowedTools "Edit,Bash(git commit *)"
done
```

3 Test on a few files, then run at scale

Refine your prompt based on what goes wrong with the first 2-3 files, then run on the full set. The `--allowedTools` flag restricts what Claude can do, which matters when you're running unattended.

You can also integrate Claude into existing data/processing pipelines:

```
claude -p "<your prompt>" --output-format json | your_command
Use --verbose for debugging during development, and turn it off in production.
```

Safe autonomous mode

Use `claude --dangerously-skip-permissions` to bypass all permission checks and let Claude work uninterrupted. This works well for workflows like fixing lint errors or generating boilerplate code.

💡 ⚠️ Letting Claude run arbitrary commands is risky and can result in data loss, system corruption, or data exfiltration (e.g., via prompt injection attacks). To minimize these risks, use `--dangerously-skip-permissions` in a container without internet access. With sandboxing enabled (`/sandbox`), you get similar autonomy with better security. Sandbox defines upfront boundaries rather than bypassing all checks.

Avoid common failure patterns

These are common mistakes. Recognizing them early saves time:

- **The kitchen sink session.** You start with one task, then ask Claude something unrelated, then go back to the first task. Context is full of irrelevant information.

o **Fix:** `/clear` between unrelated tasks

- **Correcting over and over.** Claude does something wrong, you correct it, it's still wrong, you correct again. Context is polluted with failed approaches.

o **Fix:** After two failed corrections, `/clear` and write a better initial prompt incorporating what you learned.

- **The over-specified CLAUDE.md.** If your CLAUDE.md is too long, Claude ignores half of it because important rules get lost in the noise.

o **Fix:** Ruthlessly prune. If Claude already does something correctly without the instruction, delete it or convert it to a hook.

- **The trust-then-verify gap.** Claude produces a plausible-looking implementation that doesn't handle edge cases.

o **Fix:** Always provide verification (tests, scripts, screenshots). If you can't verify it, don't ship it.

- **The infinite exploration.** You ask Claude to "investigate" something without scoping it. Claude reads hundreds of files, filling the context.

o **Fix:** Scope investigations narrowly or use subagents so the exploration doesn't consume your main context.